

1. Introduction to PPO and SAC

Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2] are two widely used deep reinforcement learning algorithms which are introduced in the late 2010s. These algorithms are used for continuous control but they differ in how they use the *data*. PPO is an on-policy method, which means that each policy iteration update is computed using trajectories collected by the current policy, and when the policy is updated all previously gathered data is discarded. SAC, on the other hand, is an off-policy method, it stores the previous transitions in a replay buffer, meaning that it can continue to learn from older transitions stored in the buffer, which usually makes it much more sample efficient.

PPO is a gradient method that seeks to take the largest possible improvement step with destabilizing the policy. It stems from the previously proposed Trust

Proximal Policy Optimization [1] is a gradient method that optimizes a clipped surrogate objective function. It was a direct improvement over the previously proposed Trust Region Policy Optimization's (TRPO) pitfalls due to the involvement of KL Divergence as a constraint over optimizing the surrogate objective function. PPO proposes the use of a clip function to prevent large policy updates instead of KL divergence penalty or constraint. "PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time"[1].

Soft Actor Critic (SAC) is an off-policy actor-critic RL algorithm based on the maximum entropy reinforcement learning framework. The soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration [2]. This was an improvement over previously proposed RL methods in terms of sample efficiency as it is an off-policy method and stability to convergence and sensitivity to hyperparameters which was notoriously tough to achieve with off-policy model free methods.

2. Relevance to Robotics

Robotics pose a specific set of challenges originating from real world settings that make using classical RL techniques infeasible. Methods like PPO and SAC mitigate these challenges quite well making them a good choice for robotics tasks. For instance, PPO and SAC can support large sizes of state space and action space and can generalize well to scenarios that these models have not encountered enough while training. These methods also work quite well with continuous action and state spaces making them an ideal choice for robotics. PPO and SAC both show a fair amount of resilience towards noisy inputs through clipping policy updates preventing large policy shifts in PPO and entropy regularized objectives reducing over-reliance on any single observation in SAC.

Even though these methods work generally well and accommodate the above mentioned challenges, it is important to note that they individually work best for certain cases. PPO is feasible and effective in simulation-driven robotics workflows, where large amounts of data can be generated cheaply. In contrast, SAC is more feasible for real-world robotics applications. Its sample efficiency, robustness to noise, and ability to learn from off-policy data make it better suited for physical systems with limited interaction budgets, however it is tough to tune and is rather sensitive to hyperparameter tuning.

3. Environment Description

- MDP formalism and why specifically half cheetah

4. PPO Hyperparameter Tuning

We chose to tune clip coefficient (ϵ) and λ (in GAE) as these affect the algorithm's working to its core.

The clip coefficient controls the policy update size, this effectively dictates how quickly or slowly the policy is moving towards the optimal. Having the correct step size is important because if the step size is too small then the policy would take a lot of timesteps to reach the optimal or else if the step size is too big the policy would end up overshooting the optimal, both cases result in the formulation of a poor policy.

Tuning lambda controls the bias-variance trade off in estimating the advantage term. As lambda moves from 0 to 1 it causes the shift in advantage estimation being carried out as TD(0) at $\lambda = 0$ (high bias) and as Monte Carlo method at $\lambda = 1$ (high variance). Temporal Difference introduces bias as it bootstraps value estimates from the immediate next step. Monte Carlo methods are unbiased but inherently show high variance, this originates from variance being accumulated over the length of the episodes. Hence, requires the agent to be run on many episodes for the value estimates to reliably converge.

The following values of ϵ were chosen: $\epsilon \in \{0.1, 0.2, 0.3\}$. We have explored values around the published default [1] ($\epsilon = 0.2$) testing change in agent behaviour under a transition from a strict ($\epsilon = 0.1$) to a permissive ($\epsilon = 0.3$) policy update constraint. For λ the following values were chosen: $\lambda \in \{0.9, 0.95, 1.00\}$. Similar strategy of exploring around the published default [1] is followed, testing change in agent behaviour as advantage estimation moves from pure monte carlo at $\lambda = 1$ to slightly towards TD(0) at $\lambda = 0.90$.

	$\lambda = 0.90$	$\lambda = 0.95$	$\lambda = 1.00$
$\epsilon = 0.1$	1471.80	1378.69	1657.81
$\epsilon = 0.2$	4449.39	2706.83	2397.13
$\epsilon = 0.3$	1131.88	4568.06	150.90

Table 1 : Episode return for different values of ϵ and λ averaged over 10 episodes

5. SAC Hyperparameter Tuning

For the purpose of hyperparameter tuning, γ , the reward discount and tau, the polyak averaging constant were tuned.

γ plays a pivotal role in dictating the agents behaviour. It controls, to describe it intuitively, the patience of the agent. Having a larger γ forces the agent to make more long sighted decisions that give more weight to future rewards, on the other hand a smaller γ forces the agents to make short sighted decisions.

τ , or the polyak averaging constant, is used in calculating a moving average of the value network weights directly contribute to Q-function learning. It weighs the contribution of current value network weights and target value network weights (previous moving average estimate). Essentially controlling the influence of immediate changes encountered as part of learning.

For hyperparameter tuning the number of training steps were reduced to 450k from 1000k(1 million).

The following values for γ were chosen: $\gamma \in \{0.90, 0.95, 0.99\}$. Here we have tested the change that arises from aggressive discounting ($\gamma = 0.90$) to long horizon planning ($\gamma = 0.99$). For τ we chose: $\tau \in \{0.05, 0.001, 0.005\}$. Here we have explored towards the upper range of τ relative to the published default [2] because exploring even lower values of τ on a limited training budget would be uninformative.

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	2294.81	2170.51	8556.36
$\tau = 0.01$	2371.91	8707.92	8552.37
$\tau = 0.05$	2385.95	2174.07	9418.79

Table 2 : Average episode return for agents with different γ and τ configurations averaged over 10 episodes

6. Results and Comparison

discuss structure and approach with sahaj

7. Proposed Robotics Task

- study POMDP

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *CoRR*, 2017, [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [2] T. Haarnoja *et al.*, “Soft Actor-Critic Algorithms and Applications,” *CoRR*, 2018, [Online]. Available: <http://arxiv.org/abs/1812.05905>